

ASSIGNMENT # 2
CS-860: ARTIFICIAL INTELLIGENCE

RESCUE ROBOT PATH PLANNING



SUBMITTED BY:

Muhammad Soban Shaukat

(Registration No: 00000538822)

**Master of Science in
Artificial Intelligence**

Fall-2025

SUBMITTED TO:

Dr. SOYIBA JAWED

Prof, CEME, NUST

Date of Submission: 15-February-2026

**Department of Computer Sciences and Engineering, CEME. National
University of Sciences & Technology (NUST), Rawalpindi (2025)**

Introduction

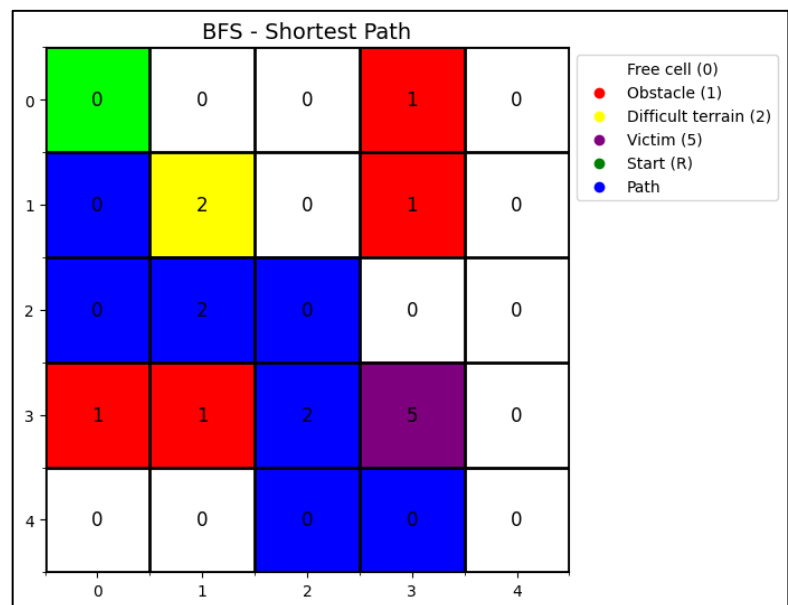
This report presents the analysis of four classical search algorithms implemented for a rescue robot path planning scenario. The robot must navigate through a 5x5 grid representing a disaster-hit building to reach a trapped victim. The building contains free cells, obstacles, and difficult terrain areas. Four algorithms were implemented and compared: Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), and A* Search. Each algorithm was evaluated based on path optimality, computational efficiency, and safety considerations.

Problem Overview

The rescue map is a 5x5 grid where the robot starts at position (0,0) and must reach the victim at position (4,4). The grid contains four types of cells: free cells (0) with traversal cost 1, obstacles (1) that cannot be passed, difficult terrain (2) with traversal cost 3, and the victim location (5). The robot can move in four directions: up, down, left, and right. All algorithms were implemented using raw Python without external libraries.

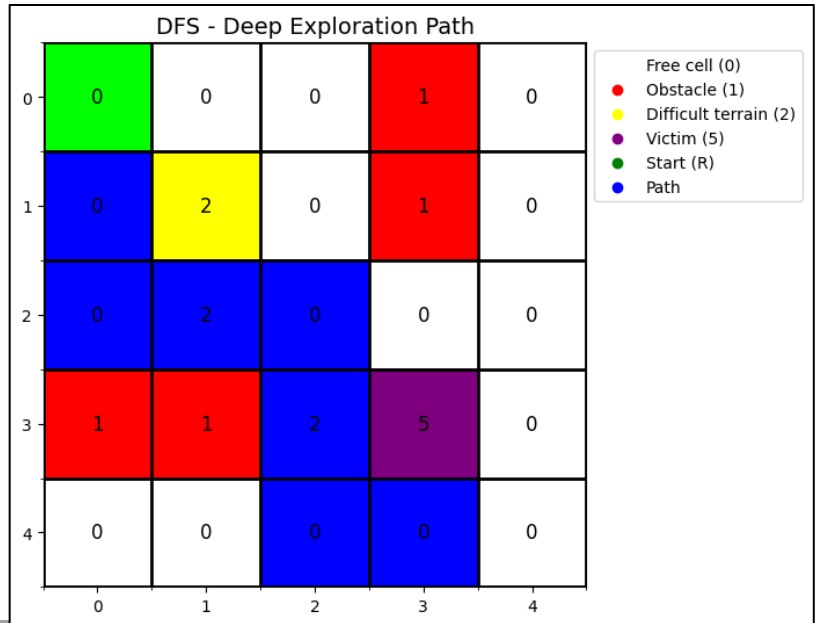
Algorithm Implementation Results

Breadth-First Search (BFS): BFS explores the grid level by level using a queue data structure. The algorithm found a path in 8 steps, exploring 20 nodes. The path was: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2) → (4,2) → (4,3) → (4,4). BFS guarantees the shortest path in steps but does not consider terrain cost, passing through difficult terrain areas.



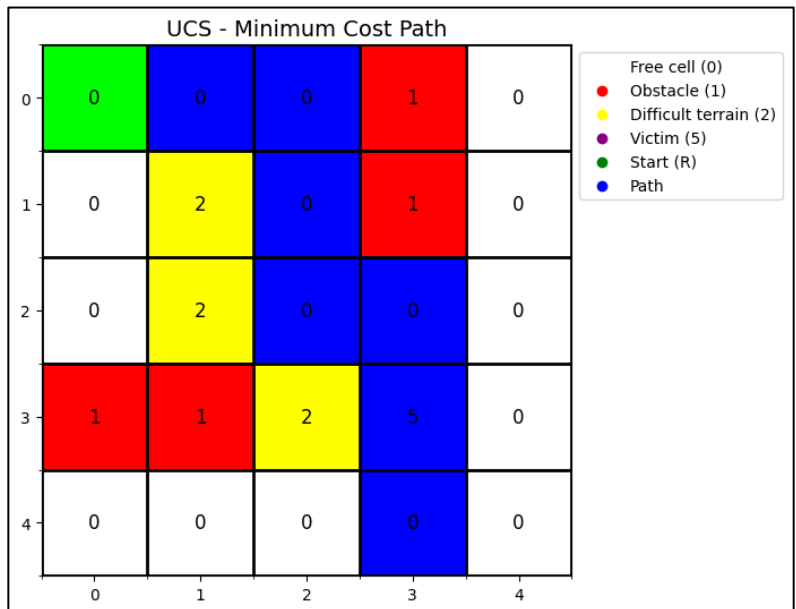
Depth-First Search

(DFS): DFS uses a stack to explore as deep as possible before backtracking. It found the same 8-step path as BFS but only explored 13 nodes. However, this efficiency was coincidental. DFS is not optimal and could find much longer paths depending on exploration order.

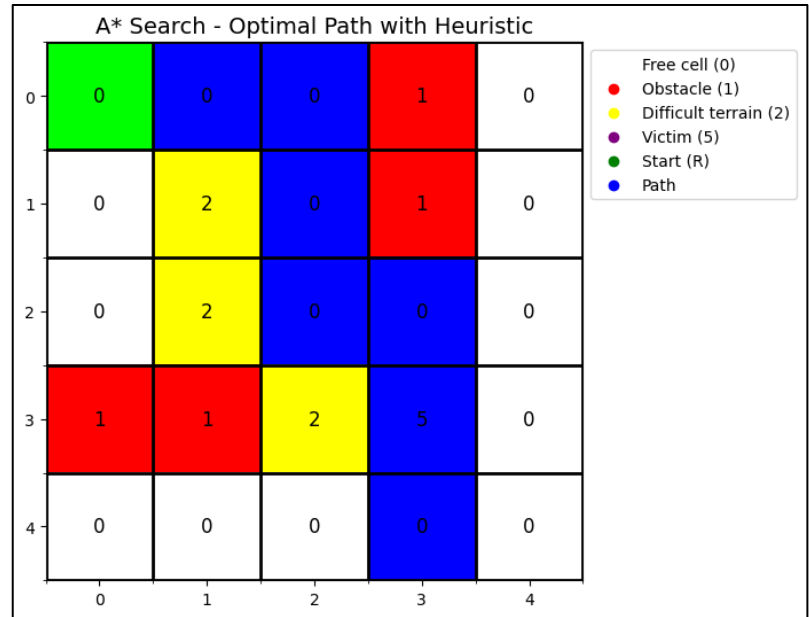


Uniform Cost Search

(UCS): UCS prioritizes paths based on cumulative cost. The path found was: $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (3,3) \rightarrow (4,3) \rightarrow (4,4)$. With 8 steps and total cost of 8, it completely avoids difficult terrain by taking the top route. UCS explored 18 nodes, making it the safest algorithm.



A* Search: A* combines cost awareness with Manhattan distance heuristic guidance. It found the same optimal path as UCS with cost 8, but explored only 13 nodes. The heuristic $h(n) = |current_row - goal_row| + |current_col - goal_col|$ guided the search intelligently, making A* both optimal and highly efficient.



Performance Comparison

Algorithm	Path Length	Total Cost	Nodes Explored	Optimal?
BFS	8	NA	20	Steps only
DFS	8	NA	13	No
UCS	8	8	18	Cost: Yes
A*	8	8	13	Cost: Yes

Analysis Questions

1. Which algorithm is fastest and why?

A* is the fastest algorithm, exploring only 13 nodes compared to BFS's 20 and UCS's 18 nodes. This represents a 35% reduction compared to BFS and 28% reduction compared to UCS. The speed advantage comes from the Manhattan distance heuristic that guides the search toward the goal intelligently. Instead of exploring all directions equally like BFS or based solely on cost like UCS, A* prioritizes nodes

that are both low-cost and closer to the goal, making it ideal for time-critical rescue operations.

2. Which algorithm is safest for rescue missions?

Both UCS and A* are equally safest with total path cost of 8. These algorithms consider terrain difficulty and actively avoid dangerous areas. Both found the top route through free cells, completely avoiding difficult terrain (cost 3). UCS and A* treat safety as a quantifiable metric through the cost function, which is critical for rescue missions where difficult terrain could represent unstable structures or fire zones. While both are equally safe, A* is faster, making it the preferred choice.

3. Why is A* preferred in real-world rescue robots?

A* is preferred because it combines optimality, efficiency, and intelligence. It guarantees the minimum cost path like UCS while being significantly more efficient by exploring fewer nodes. The heuristic guidance makes it goal-directed rather than blindly exploring all possibilities. In rescue scenarios, these factors translate to faster victim location, reduced robot exposure to hazards, lower power consumption, and better resource utilization. A* is adaptable with different heuristics for various environments. Modern rescue robots, autonomous vehicles, and drones all employ A* because it provides the best balance between optimal solutions and practical speed in time-sensitive situations.

4. What happens if the heuristic is inaccurate?

Heuristic accuracy significantly impacts A* performance. If the heuristic overestimates actual cost, it becomes inadmissible and A* loses optimality, potentially choosing suboptimal or dangerous routes. If the heuristic underestimates, A* remains optimal but becomes less efficient, exploring more nodes and behaving more like UCS. In extreme cases where $h(n)$ equals zero, A* becomes UCS. Completely wrong random heuristics cause incorrect exploration directions. Our Manhattan distance heuristic is admissible because it never overestimates, the

straight-line grid distance is always less than or equal to any actual path navigating around obstacles, making it reliable for grid-based pathfinding.

Conclusion

This assignment successfully demonstrated four classical search algorithms for rescue robot path planning. Results show that while uninformed algorithms like BFS and DFS find paths, they lack intelligence to optimize for safety. Cost-aware algorithms like UCS and A* are essential for real-world applications. A* emerged as the superior choice, combining UCS safety with superior efficiency through heuristic guidance. The Manhattan distance heuristic reduced node exploration by 28% compared to UCS while maintaining optimal path quality. These findings reinforce why A* is the industry standard for autonomous navigation in robotics, GPS systems, and emergency response applications where both speed and safety are critical.

.